

WEB TECHNOLOGIES

CSE-4004

LECTURE 16: AJAX



{JSON}

By: Dr.MD.Sirajuddin

AJAX

- **Asynchronous**
- **JavaScript**
- **And**
- **XML**
- **Web Application development technique used to create responsive web pages.**
- **Allows web pages to communicate with a web server asynchronously.**
- **With AJAX, only part of the page updates while the rest stays the same, so it feels faster and smoother.**
- **AJAX updates only what's needed, not the whole page.**

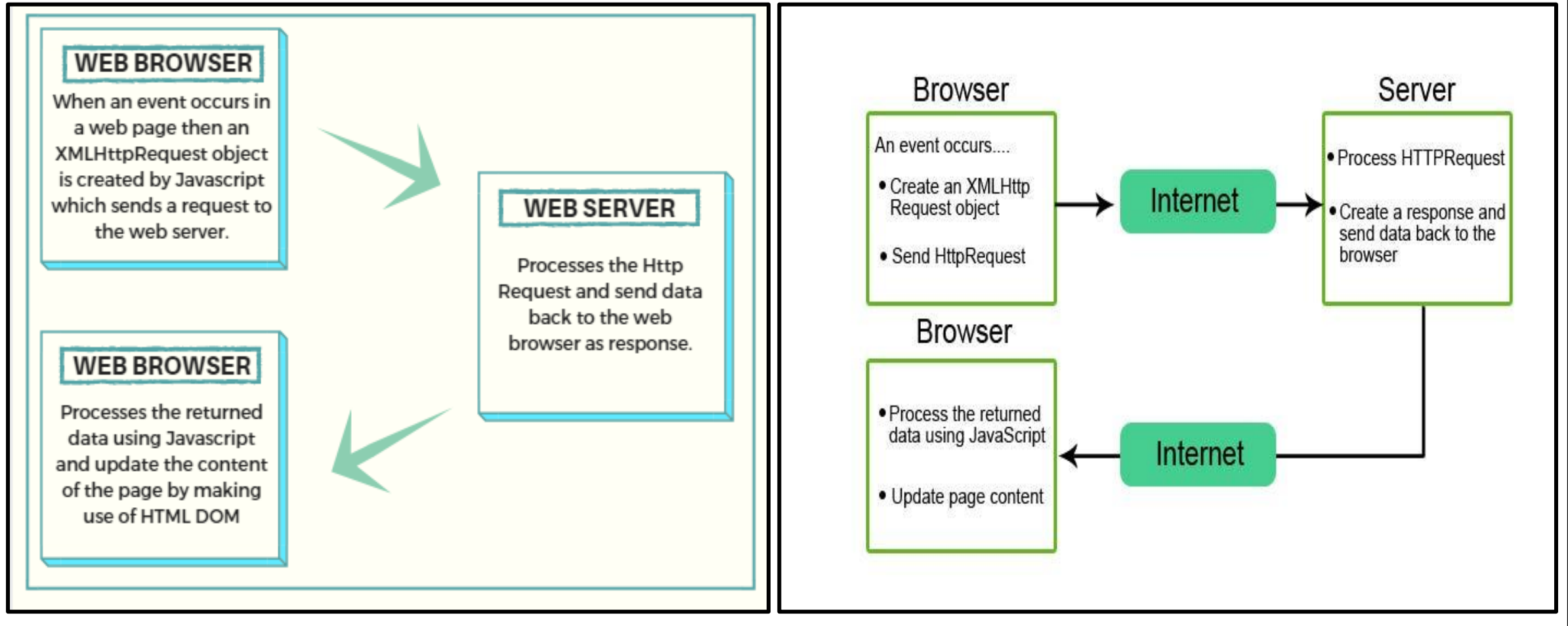
Advantages of AJAX

- Enhances user experience
- Makes web pages more interactive and responsive
- Reduces load on the Server *(Client requests are transmitted in small packets)*

Applications of AJAX

- Google Search Suggestions
- Online Form Validation
- News Feed Updation
- Updating like/comments without page reload
- Messenger / Chat Apps

- **AJAX sends a request to the server in the background, receives the response, and updates the web page without reloading it**



- For **AJAX**, the **sequence of technologies used** is usually like this:
 1. **HTML** – To create the structure of the web page.
 2. **CSS** – To style the page to make it attractive.
 3. **JavaScript** – To write the code that handles sending and receiving requests.
 4. **XMLHttpRequest / Fetch API** – The JavaScript object/method used to send requests to the server asynchronously.
 5. **Server-side language (PHP, Node.js, etc.)** – To process the request and send data back.
 6. **Data format (XML, JSON, or text)** – The format in which the server sends the response.

Creating an XMLHttpRequest Object

- All modern browsers have built in XMLHttpRequest object.
- *let variable_name = new XMLHttpRequest();*
- *let xhr = new XMLHttpRequest();*

METHODS USED

Method	Purpose	Example
open(method, url, async)	Prepares the request (method = GET/POST, URL to request, async true/false)	xhr.open('GET', 'students.json', true);
send(data)	Sends the request to the server	xhr.send(); xhr.send(JSON.stringify(student));
setRequestHeader(header, value)	Sets HTTP headers (needed for POST or JSON requests)	xhr.setRequestHeader('Content-Type', 'application/json');

PROPERTIES OF XMLHttpRequest Object

Property	Purpose	Example
responseText	Contains the response from the server as a string (text or JSON text)	let data = xhr.responseText;
responseXML	Contains the response as an XML document (if server sends XML)	let xmlDoc = xhr.responseXML;
status	HTTP status code returned by server	xhr.status === 200 means OK
statusText	Status message returned by server	console.log(xhr.statusText) e.g., "OK"
readyState	Current state of the request (0–4)	0 = UNSENT, 4 = DONE
timeout	Maximum time in milliseconds to wait for response	xhr.timeout = 5000;
withCredentials	Indicates whether cookies and credentials are sent	xhr.withCredentials = true;

readyState Property

Value	Name	Meaning
0	UNSENT	Request is created but open() not called yet
1	OPENED	open() has been called, request is ready to be sent
2	HEADERS_RECEIVED	send() called, response headers received
3	LOADING	Response is being received (partial data available)
4	DONE	Request finished and full response is available

Status Property

Status Code	Meaning
200	OK – Request succeeded
201	Created – Resource created (POST requests)
400	Bad Request – Server could not understand
401	Unauthorized – Login required
403	Forbidden – Access denied
404	Not Found – URL not found
500	Internal Server Error – Server problem

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX Simple Example</title>
</head>
<body>
  <h2>AJAX Example: Load Text on Button Click</h2>
  <button onclick="loadText()">Click Me!</button>
  <h1 id="myPara">This is where text will appear.</h1>
  <script>
    function loadText() {
      // 1. Create XHR object
      let xhr = new XMLHttpRequest();
      // 2. Configure request (GET, file, async)
      xhr.open('GET', 'data.txt', true);
      // 3. Define what happens when response is ready
      xhr.onreadystatechange = function() {
        if (xhr.readyState === 4 && xhr.status === 200) {
          // 4. Update <p> content
          document.getElementById('myPara').textContent = xhr.responseText;
        }
      };
    }
  </script>

```

```
// 5. Send the request
    xhr.send();
}
</script>
</body>
</html>
```

Web Server	Port No.
Live Server (VS Code)	5500
Python http.server	8000
Apache Tomcat	8080

EXECUTION STEPS

1. Create **ajax1.html** current directory
2. Create **data.txt** in current directory
3. Intall **Live Server** in VS Code
4. Open **ajax1.html** → Open with **Live Server**

- Data is transmitted as plaintext(String) over the network
- Not as JavaScript objects
- HTTP/1.1 200 OK
- Content-Type: application/json
- {"name":"Bob","age":22}
- This is just text inside response body.
- Client Receives: `xhr.responseText` → `'{"name":"Bob","age":22}'` as string
- Convert string → JavaScript object
- `var data = JSON.parse(xhr.responseText);`
- `console.log(data.name);`

XML Transmission

- **HTTP/1.1 200 OK**
- **Content-Type: application/xml**

<student>

<name>Bob</name>

<age>22</age>

</student>

Again → This is text.

Browser automatically parses it

var xmlDoc = xhr.responseXML;

var name = xmlDoc.getElementsByTagName("name")[0].textContent;

getElementsByTagName() always returns a collection, even if there is only one element.

[0] means → Bob (first element)

textContent → Text Inside the Tag

Difference Between JSON and XML

JSON	XML
Transmitted as text	Transmitted as text
Browser does NOT auto-parse	Browser auto-parses
Use JSON.parse()	Use responseXML
Access like object (data.name)	Use DOM methods